

1. 머신러닝과 딥러닝 / 2. 실습 환경 설정과 파이토치 기초

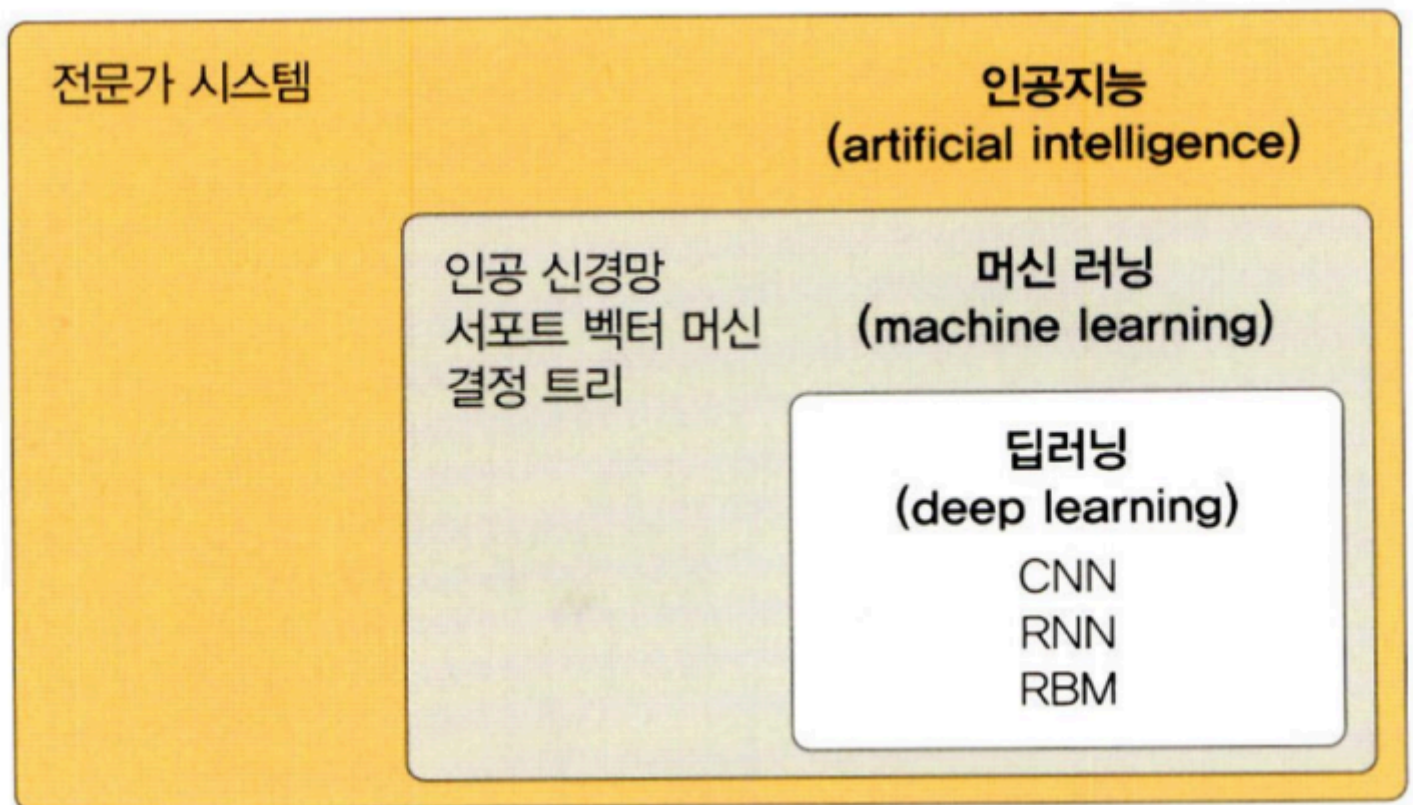
AI/데이터

2025/09/05 16:30

<http://blog.naver.com/sirutio03/223996588776>

1.1 인공지능, 머신러닝과 딥러닝

- 머신러닝: 주어진 데이터를 인간이 전처리, 범용적인 목적+데이터 특징 스스로 추출 불가
- 딥러닝: 대량의 데이터를 신경망에 적용하면 모델이 스스로 특징 추출까지 수행



구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘을 적용하여 예측을 수행한다.	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다.
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능하다.	구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용된다.
데이터	일반적으로 수천 개의 데이터가 필요하다.	수백만 개 이상의 데이터가 필요하다.
훈련 시간	단시간	장시간
결과	일반적으로 점수 또는 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

머신러닝과 딥러닝의 관계, 차이

1.2 머신러닝이란

- 머신러닝 학습 과정

: 학습 단계(훈련 데이터 알고리즘에 적용하여 학습, 모델 생성) - 예측 단계(결과 예측)

1. 데이터: 학습 모델 제작에 사용, 훈련/검증 데이터셋
2. 모델: 학습 단계의 최종 결과, 가설

- 머신러닝 학습 알고리즘

1. 지도 학습: 정답을 알려주고 학습
2. 비지도 학습: 정답을 알려주지 않고 특징이 유사한 데이터끼리 클러스터링해 학습
3. 강화 학습: 행동에 대한 보상을 받으며 학습을 진행

1.3 딥러닝이란

: 인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신 러닝 방법의 일종

- 딥러닝 학습 과정

1. 데이터 준비: 파이토치, 케라스, 캐글
2. 모델 정의: 신경망 생성, 은닉층 많을수록 성능이 좋아지지만 과적합 위험 비례함
3. 모델 컴파일: 활성화 함수, 손실 함수, 옵티마이저를 선택
4. 모델 훈련: 한 번에 처리할 데이터양 지정(에포크 선택)
5. 모델 예측: 검증 데이터셋으로 실제 예측 진행

+ 심층 신경망: 은닉층이 2개 이상인 신경망, 가중치 값을 업데이트하기 위한 역전파가 중요

• 딥러닝 학습 알고리즘

1. 지도 학습

: CNN(합성곱 신경망, 이미지 분류/인식/분할), LSTM(RNN의 기울기 소멸 문제 해결, 시계열 문제)

2. 비지도 학습

: 워드 임베딩(단어를 벡터로 표현, 자연어 처리), 군집(머신러닝의 군집과 유사)

3. 전이 학습

: 사전 학습 모델을 가지고 미세 조정을 통해 학습시킴

4. 강화 학습(머신러닝과 동일)

구분	유형	알고리즘
지도 학습 (supervised learning)	분류(classification)	• K-최근접 이웃(K-Nearest Neighbor, KNN) • 서포트 벡터 머신(Support Vector Machine, SVM) • 결정 트리(decision tree) • 로지스틱 회귀(logistic regression)
	회귀(regression)	선형 회귀(linear regression)
비지도 학습 (unsupervised learning)	군집(clustering)	• K-평균 군집화(K-means clustering) • 밀도 기반 군집 분석(DBSCAN)
	차원 축소 (dimensionality reduction)	주성분 분석 (Principal Component Analysis, PCA)
강화 학습 (reinforcement learning)	—	마르코프 결정 과정 (Markov Decision Process, MDP)

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습(unsupervised learning)	군집(clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	-	마르코프 결정 과정(MDP)

머신러닝/딥러닝 학습 알고리즘

2.1 파이토치 개요

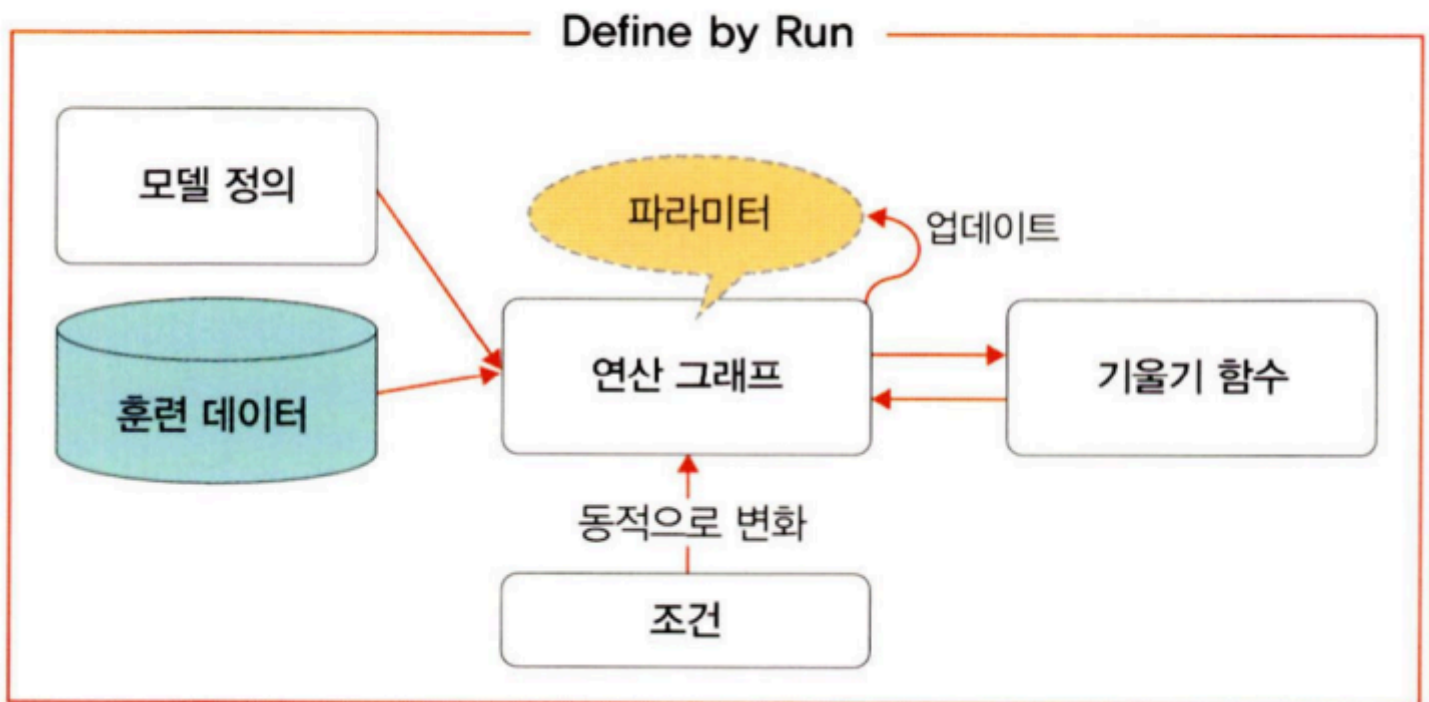
간결하고 빠른 구현(GPU 연산, 딥러닝 연구 플랫폼)

- 파이토치 특징 및 장점

: GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크, 단순함+낮은 CPU 활용+직관성

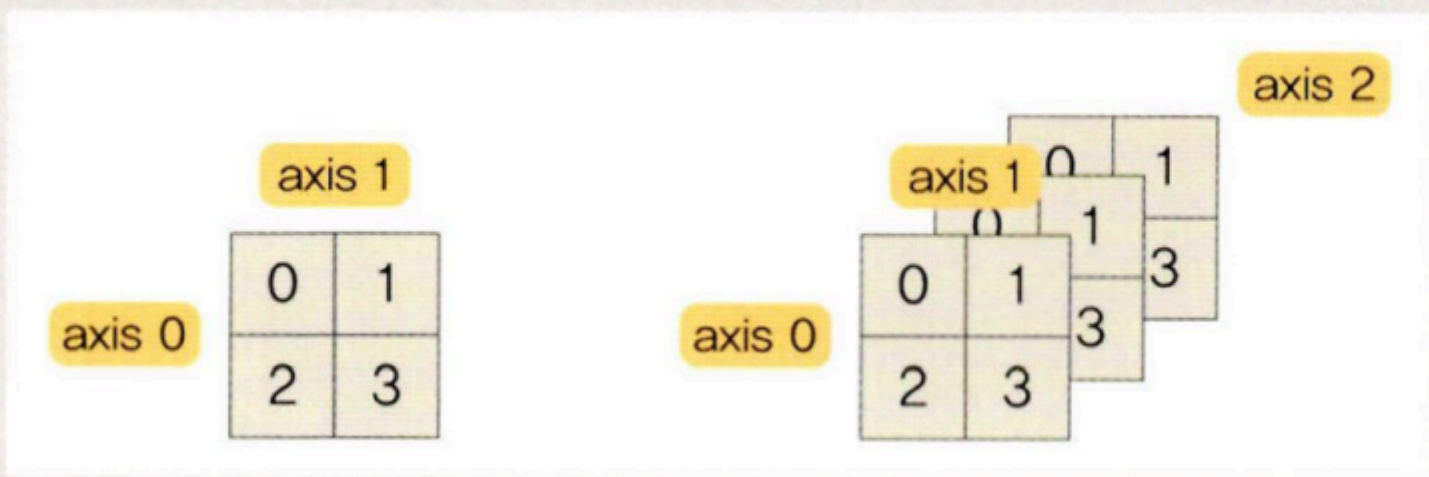
1. GPU - 연산 속도를 빠르게 함(기울기 계산 시 미분 사용, CUDA/cuDNN)
2. 텐서 - 파이토치의 데이터 형태, 단일 데이터 형식으로 된 자료들의 다차원 행렬
3. 동적 신경망 - 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망(Define By Run, 그래프와 연산 분리해서 생각할 필요 없음)

▼ 그림 2-1 파이토치 'Define by Run'



- 1차원 축(행)=axis 0=벡터
- 2차원 축(열)=axis 1=행렬
- 3차원 축(채널)=axis 2=텐서

▼ 그림 2-2 벡터, 행렬, 텐서



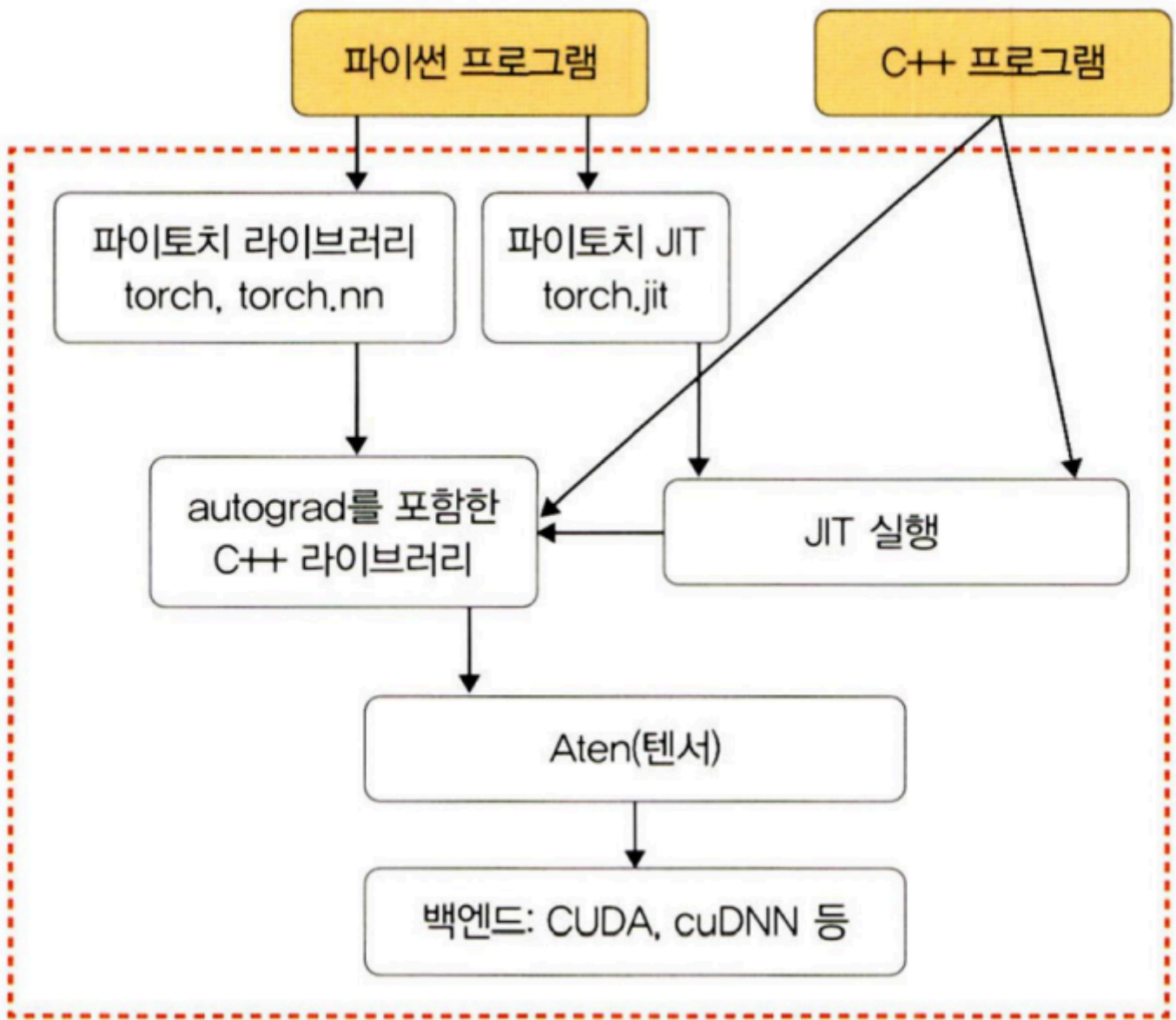
파이토치 Define by Run 방식 / 벡터, 행렬, 텐서

+ 연산 그래프란? : 노드와 엣지로 이루어진 그래프. 노트는 변수, 텐서이고 엣지는 연산 담당이다.

- 파이토치의 아키텍처

1. 파이토치 API: 실제 계산 수행 X, 사용자 편의 위해 패키지 제공됨 (torch/GPU 지원 텐서, torch.autograd/자동 미분, torch.nn/신경망 구축 및 훈련, torch.multiprocessing/파이썬 멀티프로세싱, torch.utils/기타 유틸리티 제공)
2. 파이토치 엔진

▼ 그림 2-5 파이토치 엔진



3. 텐서 연산: 상위 API에서 할당된 거의 모든 계산 수행

+ 텐서를 메모리에 저장하기

: 1차원 배열 형태여만 저장 가능(스토리지), 오프셋과 스트라이드 개념으로 전치 행렬 구분함

2.2 파이토치 기초 문법

• 텐서 다루기

1. 텐서 생성 및 변환

`print(torch.tensor([[1,2],[3,4]]))` ----- 2차원 형태의 텐서 생성 `print(torch.tensor([[1,2],[3,4]], device="cuda:0"))`
----- GPU에 텐서 생성 `print(torch.tensor([12], [3,4]), dtype=torch.float64))` ----- dtype을 이용하여 텐서 생성

2. 텐서의 인덱스 조작

> 텐서 자료형

`torch.FloatTensor`: 32비트의 부동 소수점

`torch.DoubleTensor`: 64비트의 부동 소수점

`torch.LongTensor`: 64비트의 부호가 있는 정수

`temp = torch.FloatTensor([1, 2, 3, 4, 5, 6, 7])` ----- 파이토치로 1차원 벡터 생성 `print(temp[0], temp[1], temp[-1])` ----- 인덱스로 접근 `print(temp[2:5], temp[4:-1])` ----- 슬라이스로 접근

3. 텐서 연산 및 차원 조작

타입이 다르면 연산 불가, 그외 수학 연산 가능 / `view`, `stack`, `cat`, `t`, `transpose` 등으로 차원 변경

`V = torch.tensor([1, 2, 3])` ----- 길이가 3인 벡터 생성 `w = torch.tensor([3, 4, 6])` `print(w - V)` ----- 길이가 같은 벡터 간 뺄셈 연산 # `temp`가 2*2 행렬일 때 `print(temp.view(1,-1))` ----- -1은 (1, ?)와 같은 의미로 다른 차원으로부터 해당 값을 유추, (1,4)

• 데이터 준비

1. 단순히 파일 불러와서 사용: 판다스 라이브러리 이용
2. 커스텀 데이터셋 제작 후 사용: 대량의 데이터를 조금씩 불러와 사용
3. 파이토치에서 제공하는 데이터셋 사용: 토치비전 사용

• 모델 정의

: 모듈을 상속한 클래스 사용, 계층(모듈/모델을 구성하는 한 개의 계층)|모듈(1개 이상의 계층)|모델(최종)

1. 단순 신경망 정의: 매우 단순한 모델 제작 시 사용
2. `nn.Module()` 상속하여 정의: `__init__()`에 모듈 정의, `forward()`에 연산 정의
3. `Sequential` 신경망 정의: 모델의 계층이 복잡할수록 효과 뛰어남

4. 함수로 신경망을 정의: Sequential과 동일하나 변수에 저장해 놓은 계층들을 재사용할 수 있다는 장점 O

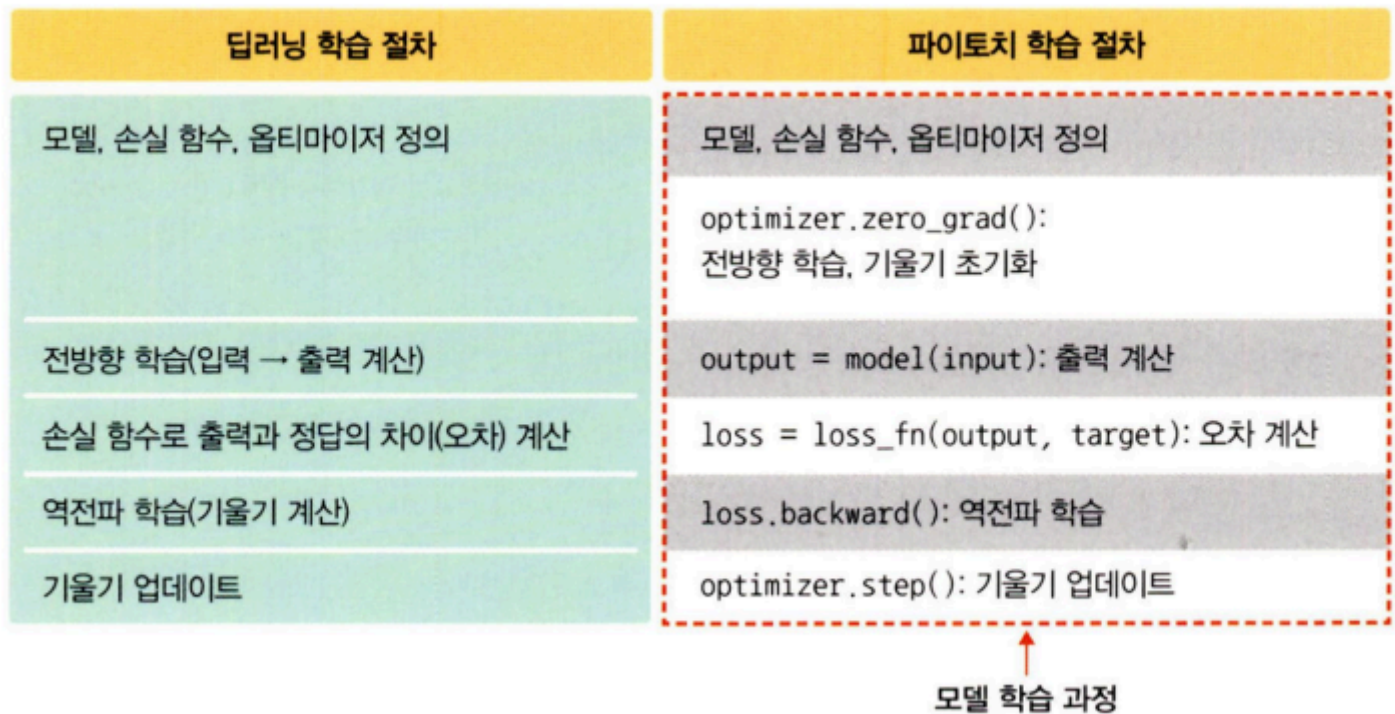
- 모델 파라미터 정의

1. 손실 함수: 학습하는 동안 출력과 정답 사이 오차 측정, BCELoss/CrossEntropyLoss/MSELoss 등 사용
2. 옵티마이저: 데이터와 손실함수 바탕으로 모델 업데이트 방법 결정
3. 학습률 스케줄러: 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소(decay)시켜 줌
4. 지표: 훈련과 테스트 단계를 모니터링

- 모델 훈련

$y = wx + b$ 라는 함수에서 w 와 b 의 적절한 값을 찾는 학습 진행(w, b 는 파라미터)

▼ 그림 2-12 파이토치 학습 절차



모델 학습 절차

- 모델 평가

1. 함수를 이용해 평가
2. 모듈을 이용해 평가

- 훈련 과정 모니터링

: 텐서보드 이용 시 각종 파라미터 값의 변화를 시각화하여 살펴볼 수 있음(설정-기록-구조 관찰)

2.3 실습 환경 설정

- 아나콘다 설치
- 가상 환경 생성 및 파이토치 설치
- 주피터 노트북 설치

2.4 파이토치 코드 맛보기

예제 데이터셋 분포 비율 확인(시각화)

- 데이터를 범주형 타입으로 변환
1. 범주형 데이터 -> dataset[category] -> Numpy array -> 텐서 : 범주형 데이터 넘파이 배열로 바꾸기 위해 cat.codes 사용
 2. np.stack()으로 두 넘파이 객체 합침

+ np.stack()과 np.concatenate()

- 레이블로 사용할 칼럼을 텐서로 변환
1. get_dummies()로 가변수 변환

+ ravel()/reshape()/flatten() : 텐서 차원 변환

- 워드 임베딩을 위해 범주형 칼럼을 N차원으로 변환
 - 데이터셋 분리(훈련용/테스트용)
 - 모델 네트워크 생성
1. nn.Module() 상속
 2. __init__ 매개변수: embedding_size, output_size, layers, p
 3. 각 계층 for문 사용해서 추가
 4. forward()로 연산 시행
- 모델 객체 생성
 - 파라미터 정의
 - CPU/GPU 사용 지정
 - 모델 학습
 - 모델 예측 확인
 - 테스트 데이터셋 이용해서 정확도 확인

: 정확도, 재현도, 정밀도, F-1 Score